
Test-Time Compute Allocation: Search vs. Verify

Author Name

Affiliation

The Concept This page is the course’s entry point: it defines the concept below, states the learning objectives, links the frontier papers, and routes you to the video, slides, and notebooks.

The concept taught in this course is test-time compute allocation: given a fixed inference budget (a fixed number of model calls), how should a model divide it between search and verification? Search generates many candidate answers and aggregates them, for example by majority voting. Verification generates fewer answers and checks each one with a judge (a verifier). Search carries a mathematical guarantee: with independent candidates and accuracy p above $1/2$, the failure probability of majority voting falls exponentially in N ; verification has no such guarantee, and only helps when the judge can tell good answers from bad ones. In this course the judge is the model grading its own answers; our measurements show it is weak, barely separating correct from incorrect candidates. Under a weak judge and a moderate budget (the setting of this course), search usually wins; the best split depends on your model, your judge, and your budget. Our measured setting (100 MATH 500 problems, 50 calls per problem, Qwen3-4B, three seeds) is in Table 1.

Table 1: Measured accuracy on the 100-problem MATH 500 subset (Qwen3-4B, three seeds, budget 50 calls per problem).

Strategy	Split	Accuracy [95% CI]	Reference
Search	50 candidates, majority vote	0.853 [0.81, 0.89]	[4] (Majority)
Search + light verification	25 candidates $\times$ 1 check	0.717 [0.67, 0.77]	–
Verification	5 candidates $\times$ 9 checks	0.753 [0.70, 0.80]	[4] (PRM weighted)

Leveling and Prerequisite Knowledge Level 1 (no prerequisites): watch the five-minute video and read this page; you get the core trade-off in plain terms. Level 2 (systematic): work through the notebooks and slides; this path is aimed at graduate students and early-career researchers in LLM reasoning and is adaptable for advanced undergraduates. Before starting, you should answer: (1) Why does an LLM reload all weights for every token? (2) If each candidate is right 30% of the time, what is the chance that majority voting over three independent candidates is right? (3) What do MATH and GSM8K measure, and how do their difficulties differ? Answers are built into Notebook 00.

Learning Objectives By the end of this material, learners will be able to: **Level 1:** explain the search–verification trade-off in plain language, naming the two conditions under which search wins (a weak judge, a moderate budget); evidence: a 30-second retelling. **Level 2:** (1) estimate a model’s failure rate with a simplified majority-voting model and predict the candidate count N needed; evidence: a calculation. (2) plot their own budget–accuracy curve with at least two data points and read the six-model curve family; evidence: a plot plus a written reading. (3) given a new task and a fixed budget, reason out the best search–verify split from the model’s accuracy and the judge’s strength; evidence: an analysis report. (4) advanced: judge whether a research paper’s baselines are fair; evidence: a critical report.

Linked Papers Test-time compute allocation is in active use at the frontier; each paper below builds on it. *Scaling LLM Test-Time Compute Optimally* [4] studies the allocation problem directly: compute-optimal splits beat fixed best-of- N by up to $4\times$, and with a strong judge (e.g., a trained reward model), verification-guided search wins at low to moderate budgets. *Sample, Scrutinize and Scale* [5] pushes the allocation toward verification at very large budgets, where sampling-based search with self-verification keeps scaling. *Let Me Think!* [2] allocates the budget between one long chain and many short ones, showing serial expansion can be exponentially better. *Provable Scaling Laws*

[1] gives pairwise-comparison aggregation (a search-style strategy) a guarantee even below $p = 1/2$. *When to Solve, When to Verify* [3] asks the same allocation question with a generative reward model, finding self-consistency (search) more compute-efficient than verification for most budgets, which is exactly the weak-judge regime that this course measures.

Teaching Materials Summary All materials are original and were created for the NeurIPS 2026 Education Track; everything lives in <https://github.com/Solitus0726/search-vs-verify-course>. Suggested order: start with the self-check (Notebook 00) to find your path, then follow its materials in the order shown in Figure 1, with the slides as a reference for the math. The learning path in Figure 1 routes you through these materials. The video (about five minutes) explains the trade-off in plain terms. Notebook 00 runs the three-question self-check, and failing any question routes you to the supplementary materials. Notebook 01 plots budget–accuracy curves and reads the six-model family (p from 0.37 to 0.73); Notebook 02 finds the optimal split under the budget identity $N \times (1 + M) = 50$; Notebook 03 predicts the best split for GSM8K and tests it; Notebook 04 is the report template. The slides and strategy diagram add the formal math and a one-page visual summary. The scripts and data make every result reproducible: about one to two hours from scratch, or instantly with the pre-computed cache included in the repository.

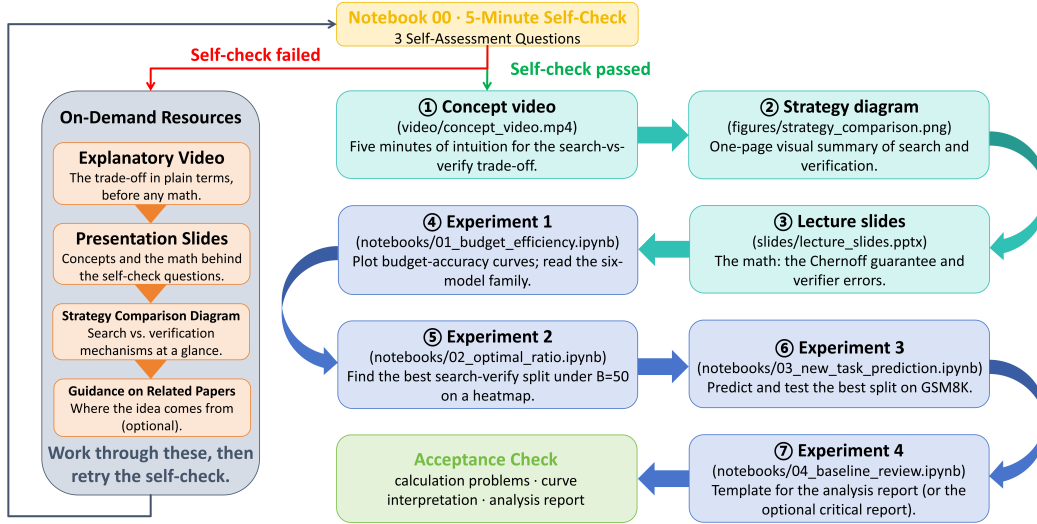


Figure 1: Learning path: a five-minute self-check routes you to supplementary materials or to one of three material sequences (intuition-first, systematic, quick reference), ending at the acceptance checks.

References

- [1] Yanxi Chen, Xuchen Pan, Yaliang Li, Bolin Ding, and Jingren Zhou. Provable Scaling Laws for the Test-Time Compute of Large Language Models. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=GBMzJLhsRj>.
- [2] Parsa Mirtaheri, Ezra Edelman, Samy Jelassi, Eran Malach, and Enric Boix-Adserà. Let Me Think! A Long Chain of Thought Can Be Worth Exponentially Many Short Ones. In D. Belgrave, C. Zhang, H. Lin, R. Pascanu, P. Koniusz, M. Ghassemi, and N. Chen, editors, *Advances in Neural Information Processing Systems*, volume 38, Main Conference, pages 166795–166826. Curran Associates, Inc., 2025. doi: 10.52202/085713-5561. URL https://proceedings.neurips.cc/paper_files/paper/2025/file/f3b336ac87912786ef2d72238058cb4f-Paper-Conference.pdf.
- [3] Nishad Singhi, Hritik Bansal, Arian Hosseini, Aditya Grover, Kai-Wei Chang, Marcus Rohrbach, and Anna Rohrbach. When To Solve, When To Verify: Compute-Optimal Problem Solving and Generative Verification for LLM Reasoning. In *Second Conference on Language Modeling*, 2025. URL <https://openreview.net/forum?id=R7qRUFHGTx>.
- [4] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM Test-Time Compute Optimally Can be More Effective than Scaling Parameters for Reasoning. In Y. Yue, A. Garg, N. Peng, F. Sha, and R. Yu, editors, *International Conference on Learning Representations*, volume 2025, pages 10131–10165, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/file/1b623663fd9b874366f3ce019fd444-Paper-Conference.pdf.

- [5] Eric Zhao, Pranjal Awasthi, and Sreenivas Gollapudi. Sample, Scrutinize and Scale: Effective Inference-Time Search by Scaling Verification. In Aarti Singh, Maryam Fazel, Daniel Hsu, Simon Lacoste-Julien, Felix Berkenkamp, Tegan Maharaj, Kiri Wagstaff, and Jerry Zhu, editors, *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 77272–77309. PMLR, 13–19 Jul 2025. URL <https://proceedings.mlr.press/v267/zhao25a.html>.